

Ontology-Driven IaC & AI

The Beginning of the End for YAML as We Know It

A new paradigm for Infrastructure-as-Code,
powered by AI and Ontology-Driven DevOps

Presenter: Sacha Lin



AI & DevOps Journey

 Powered by XDevOps.AI

A community exploring how AI is reshaping the DevOps journey — from automation to modernization — through open discussion & shared learning.



Understand New Tools

Dive deep into AI & DevOps integrations



Share Experience

Learn from real-world success & failures



Debate AI Risks

Discuss safety, ethics, and limits of AI-driven automation



Document the Journey

Capture lessons & progress for the community

Perfect For:

-  DevOps, SRE, and Platform Engineers
-  Software Developers interested in automated deployments
-  AI Enthusiasts & Researchers curious about AI use cases in DevOps

Vendor-Neutral | Community-Driven | Knowledge Sharing



The Promise

“By the end of this talk, you’ll see
infrastructure not as files,
but as a knowledge graph — and
understand why that changes everything.”



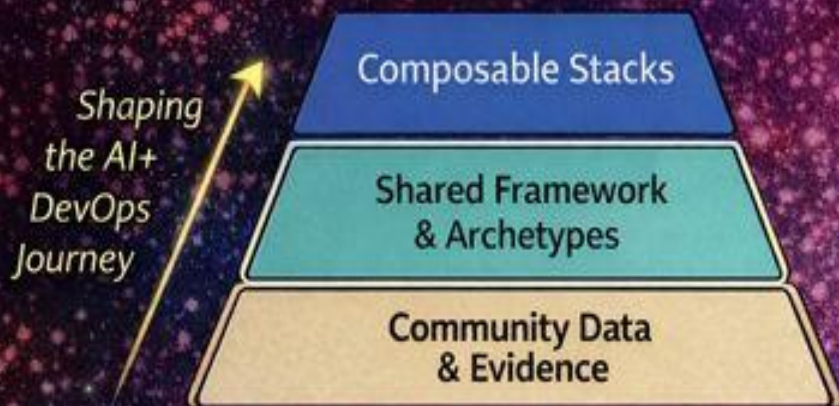


Contribute to the DevOps & SRE AI Atlas 2025



A community knowledge base to define AI use cases, risks, and patterns for DevOps/SRE.

- What AI use cases are valid & valuable?
- How do we guardrail & validate safely?
- What patterns & lessons learned apply?



Share & map AI use cases that help DevOps / SRE



Document risks, failure stories & guardrails



Propose universal patterns, lessons learned, & anti-patterns



Openly discuss, debate, & improve guidelines

github.com/xdevops-ai/devops-sre-ai-atlas-2025



The Big Shift

Infrastructure became **dynamic**.
Management stayed **static**.

Modern infrastructure is:

-  Multi-cloud
-  Identity-heavy
-  Policy-driven
-  Event-driven
-  Constantly drifting
-  Deeply interconnected

But we still manage it with:

-  Static YAML, templates, and files
-  Hidden drift
-  Fragile systems
-  Slow response
-  Tech debt

✗ Static YAML was never designed for this kind of system.



How We Do It Better

Instead of managing infrastructure as files,
we manage it as knowledge.

Infrastructure becomes:



Semantic

resources have meaning



Connected

relationships are explicit



Stateful

desired vs observed is tracked



Governed

policies are first-class



Explainable

every change has context

The new approach:



Ontology

defines what the system is



AI agents

reason, validate, and act safely

The previous state

The result:



no hidden drift



no tribal knowledge



no blind changes



faster resolution

🧠 Why YAML Is Breaking Down

✖ Inherent limitations

- 🧠 No semantics
- 🔗 No relationships
- 😞 No reasoning
- 🚫 No constraints
- 🕒 No real-time awareness

🔥 Operational consequences

- 🌀 Drift everywhere
- 🔧 Fragile troubleshooting
- 📞 L1 → L2 → L3 escalation overload
- 💡 Massive technical debt
AI initiatives stall



“But what if the real problem isn’t any of these?
What if it’s this...”

🔥 PAIN #0 — Infrastructure Is a Black Box

● The real problem

- 🧠 Teams don't see the system
- 🧩 Dependencies are invisible
- 📁 Context is scattered across tools
- 🧠 Understanding lives in people's heads

★ What this causes

- fragile decisions
- slow debugging
- risky changes
- constant escalations

● The shift we need

Infrastructure must become explainable.

➡ 🚀 Not just deployed

➡ 🔍 Not just monitored

📖 But understood

! Everything else we'll discuss today is
a consequence of this single pain.

The 5 Core Pains

PAIN #1 — You can't query your infrastructure

-  "What depends on this?"
-  "Who created this?"
-  "What changed last night?"

PAIN #2 — Drift is detected too late



Batch checks



Problems found after impact

PAIN #3 — Policies are bolted on



External tools



No lifecycle for exceptions

PAIN #4 — No provenance or causality

PAIN #5 — Support escalation spiral



Context trapped in chat & tribal knowledge

L1 → L2 → L3 overload

The New Paradigm

Ontology-Driven IaC

Instead of managing infrastructure as files, we manage it as knowledge.

Infrastructure becomes:

-  **Semantic** — resources have meaning
-  **Connected** — relationships are explicit
-  **Stateful** — desired vs observed is tracked
-  **Governed** — policies are first-class
-  **Explainable** — every change has context

The foundation

-  **Ontology** → defines what the system is
-  **Knowledge Graph** → represents the live system
-  **AI agents** → reason, validate, and act safely

✨ **YAML doesn't disappear.** It becomes an output, not the source of truth.

🌟 **Infrastructure should be understood, not just deployed.**



The Self-Healing Infrastructure Loop

The Continuous Reconciliation Engine

Infrastructure no longer stops at deployment.
It continuously reconciles itself.



- ★ state is remembered.
- ★ changes are explained
- ★ drift is expected, not feared

| This is the difference between automation and autonomy.



Fixing the Support Escalation Crisis

From firefighting to system stewardship

● Today (the reality)

- 🌐 Tribal knowledge everywhere
- 🕶️ Invisible dependencies
- ⚠️ Risky commands
- 📞 Escalation waterfall
- 💣 Burnout & growing tech debt

● With Ontology + AI

- 🌌 Full system context in the graph
- 📖 AI explains root cause
- 🛡️ Role-safe operations
 - L1 → read-only
 - L2 → safe remediation
 - ➡️ L3 → full control (validated).
- 🔄 Proactive, not reactive support
- ⬇️ 50–80% fewer escalations

! We're not just changing how we write configs.
We're changing how we do support.

⚡ The 10× Advantages

What this engine unlocks

- 🔴 Fewer support escalations
Context + AI guidance reduces L1 → L3 **overload**
- 🔵 End-to-end traceability
Every change is explainable
Intent → Plan → Command → Event → Resource
- 🛠️ Drift-aware, self-healing infrastructure
Drift is detected and reconciled continuously
- 🤖 AI agents become safe actors
Validation, guardrails, and policies before execution
- ☁️ True multi-cloud unification
One semantic model across environments

✨ These aren't optimizations.
They change how infrastructure operates.



Demo — YAML vs Knowledge Graph

From files to understanding
YAML (today)

- 📖 Hundreds of lines
- 🚫 Ignored ops & inputs
- 🧐 Hard to reason about
- 🗂️ Context scattered across files

Knowledge Graph (new model)

- 🔗 Explicit relationships
- 📶 Live state (no guesswork)
- 🛡️ Policies (and lineage) attached
- 🔍 Dive in (easy SPARQL)

```
PREFIX <def>, "http://def/myn360.paloaltonosp"
PREFIX <ref>, "https://www.googleapis.com/t3872-def-system/api>

SELECT ?vmName ?nsgName ?ruleName ?priority ?port ?access

1. #1 Start at the VM
   ?vm def:refRuleToVm ?ruleName ;
   def:refVmName ?vmName ;
   FILTER(?vmName = "test01")

2. #2 Get the NG and then the NSG
   ?nsg ref:refNetworkSecurityGroup ;
   ?nsgName ?nsgName ;
   ?nsg def:refCloudResource ?ruleName .

3. #3 List individual NSG rules
   ?rule def:refRuleGrantToNSFG ;
   ?ruleName ?ruleName ;
   ?rule def:directionPortBange ?port ...
OPTIONAL { ?rule def:directionPortRange ?port . }

* Let's ask the graph!
```

! This is the moment where infrastructure
becomes interactive knowledge.



What's Ready vs What's Emerging



Ready today

- Read-only analysis
- Drift detection
- Impact analysis
- Explainability
- Policy validation



Emerging

- Autonomous remediation
- Cross-cloud orchestration
- Full NL-driven change approval loops



Where YAML Fits Now

The graceful demotion
YAML does not disappear.

 YAML becomes:

 An output format

 A GitOps compatibility layer

 A debugging artifact

✱  YAML is no longer:

 The source of truth

 The architectural model

 The memory of the system

✱  Humans stop writing YAML from *knowledge*.



What This Is NOT

- ✗ Not ChatGPT running 'kubectl'
- ✗ Not free-form AI improvisation
- ✗ Not replacing human intent
- ✗ Not skipping validation or policy

AI is the interface.

The system remains deterministic.

? Q&A

💬 Questions

🌍 Challenges from your own infra

🤖 Thoughts on AI-driven operations

🌱 Where this could (or couldn't) work for you

🌟 Let's discuss.



Contribute to the AI SRE / DevOps Atlas

A shared, open effort to map AI use cases, risks, and patterns in DevOps & SRE.

- ✓ AI use cases in DevOps / SRE
- ✓ Failure modes & risks
- ✓ Guardrails & constraints
- ✓ Patterns that **don't** work
- ✓ Real-world lessons learned

This is not a product roadmap.

It's a shared knowledge base for the community.



github.com/xdevops-ai/devops-sre-ai-atlas-2025



Help Stress-Test the Model

Share your:

- ✓ Feedback on the ideas & live demo
- ✓ Real failure stories & edge cases
- ✓ Environments you'd never trust AI with
- ✓ Situations where this approach breaks

If you want **hands-on** access to experiment, explore, or break things, we can provide **limited** access to the XdevOps.ai portal.

Email feedback@xdevops.ai





What's Next

👉 Join the next sessions & meetups

Deep dives on ontology, AI agents, and autonomous infrastructure

👉 Get hands-on

See real demos · ask hard questions · explore the model

👉 Be part of the conversation

We're shaping a new way to operate infrastructure — together



xdevops.ai

✨ This is not a finished product.

It's a movement — and it's just starting.

